

目 录

1	毕业设计（论文）内容概述	2
1.1	项目来源及开发目的和意义	2
1.2	主要开发任务	3
1.3	本人所承担任务（模块）说明	3
2	已完成的研究工作及成果	4
2.1	预定计划的执行情况	4
2.2	系统总体设计	4
2.3	系统架构设计	6
2.4	模块交互设计	9
2.5	系统的详细及实现设计	10
3	后期拟完成的研究工作及进度安排	12
3.1	后期拟完成的工作	12
3.2	后期工作的进度安排	12
4	存在的问题与困难	12
5	论文按时完成的可能性	12

一 毕业设计（论文）内容概述

1.1 项目来源及开发目的和意义

本课题来源于面向蛋白质设计任务的多 Agent 系统工程实践需求。随着大语言模型在任务规划、工具调用和复杂流程编排中的能力不断增强，使用自然语言驱动多步骤科研任务已经具备了较强的可行性。在蛋白质设计场景中，这种能力尤其具有吸引力，因为该类任务通常同时涉及序列生成、结构预测、质量评估、候选筛选和结果汇总等多个异构环节，传统流程往往依赖研究人员在不同工具之间进行人工切换、参数调整和结果比对，工作链路长、重复劳动多，而且缺少统一的状态管理和审计机制。如何将大语言模型的规划能力与生物信息工具链的可执行能力结合起来，并通过多 Agent 协作方式把规划、执行、安全审查和总结等异构职责稳定分离，成为本课题的直接出发点。

从现有系统形态来看，通用 LLM Agent 在开放环境中已经能够完成一定程度的工具调用与任务分解，但若直接迁移到蛋白质设计场景，仍然存在若干明显不足。首先，许多代理系统依赖隐式对话状态或单轮推理结果推进任务，缺乏清晰的外显状态表达，一旦执行中断、工具失败或人工需要介入，系统难以准确恢复到可继续运行的上下文。其次，科研工作流往往并非一次规划即可稳定执行到底，局部步骤失败、结果不满足约束、工具输出格式异常等情况都很常见，如果没有结构化的失败恢复机制，系统就容易在错误后陷入无序重试甚至整体崩溃。再次，蛋白质设计涉及安全性、合规性和结果可信度问题，单纯依靠模型自发判断缺乏足够的可解释性，也难以支撑后续的复盘与评估。因此，本课题并不把目标简单设定为“让模型调用更多工具”，而是更强调在复杂科研任务中建立一套可控、可恢复、可审计的智能体执行闭环。

基于上述问题，本课题拟设计并实现一个以多 Agent 协作为核心的蛋白质设计系统，以显式有限状态机作为全局控制骨架，以 Planner、Executor、Safety 和 Summarizer 四类 Agent 作为职责边界，以 ToolKG 和工具适配层作为执行基础，使系统能够围绕候选生成、状态推进、失败恢复和人在环决策形成统一流程。与单一路径执行的传统代理不同，本系统将关键决策节点明确暴露出来，通过候选集、门禁逻辑和待确认状态承载人工监督；在执行阶段通过分层恢复策略将 retry、patch 和 replan 区分为不同层级的控制动作；在结果沉淀阶段进一步通过事件日志、任务快照和报告产物保留执行证据。这一思路的意义在于，它不仅能够提高蛋白质设计任务的自动化程度，也使“多 Agent 驱动”不再停留于模块命名，而是具体落实为职责分明、可控协作的工程结构，为后续实验评估、过程回放和论文写作提供了稳定基础。

从论文选题的角度看，本课题兼具工程实现价值与研究验证价值。一方面，它试图回答“多 Agent 驱动的蛋白质设计系统应如何建立可控执行机制”这一问题，强调角色分工、状态机、人机协同和恢复控制在复杂科研任务中的必要性；另一方面，它也以蛋白质设计这一具有明确工具链和质量约束的应用场景为落点，为后续纵向机制增量实验和横向代理范式比较提供了可实现、可观察的系统载体。因此，本课题并不是对现有模型能力的简单包装，而是围绕科研场景中“如何安全、稳定、可追踪地用好多 Agent 系统”这一核心问题展开的系统性探索。

1.2 主要开发任务

围绕课题目标，中期阶段的主要开发任务可以概括为系统架构落地、关键控制链实现、领域 workflow 接入以及阶段性验证材料整理四个方面。首先，在总体架构层面，需要完成任务对象、执行计划、步骤结果和最终结果等核心数据契约的定义，并在此基础上建立 API 入口、工作流编排模块、日志与快照存储模块之间的协同关系，使系统能够以统一的数据结构驱动规划、执行和汇总全过程。其次，在智能体职责设计方面，需要将 Planner、Executor、Safety 和 Summarizer 的边界落实到实际代码中，避免规划、执行、审核和总结之间出现角色混淆，特别是要确保状态变更只能由工作流控制逻辑推进，执行代理不能绕过决策机制直接修改关键状态。

在核心运行机制方面，本课题需要完成显式 FSM 及其等待状态语义的实现，使任务可以按照创建、规划、等待确认、执行、总结和完成等阶段稳定推进。与此相配套的，是候选集决策与分层恢复机制的落地：系统需要在初始规划、局部修补和再规划等关键节点输出结构化候选，并根据风险、成本和置信度触发自动执行或人工确认；在执行链路中，还需要建立严格的恢复顺序，使单步失败优先进入有限重试，再视情况触发 patch，最后才进入 replan。为了支撑这一过程，项目还需要实现 PendingAction、Decision、EventLog 和 TaskSnapshot 等机制，使等待中的任务能够被审查、被恢复、被回放。

在领域能力接入方面，课题需要将蛋白质设计任务拆解为可执行的多阶段 workflow，并逐步完成与 ToolKG、工具适配器和远端服务的衔接。目前系统已经围绕序列探索、结构映射、质量门禁、结构条件精修、多目标打分和恢复控制形成了较为清晰的六阶段组织方式。为了让这些环节真正服务于中期论文写作，还需要进一步整理验证报告、实验设计文档、图示材料和展示路径，形成能够支撑“系统已基本建立工程闭环，但实验比较仍在持续完善”这一阶段结论的证据包。换言之，中期前的任务并不只是完成若干分散功能点，而是要把架构、实现、验证和写作材料整理成一个相互对应的整体。

1.3 本人所承担任务（模块）说明

在本课题的实施过程中，本人承担的工作主要集中在系统总体方案细化、核心 workflow 实现、控制机制落地以及阶段性材料沉淀四个方面。首先，在系统设计层面，本人围绕多 Agent 架构、显式状态机和六阶段蛋白质设计 workflow 持续整理设计文档，将任务生命周期、角色职责、数据契约和恢复顺序固化为较为明确的系统约束，为后续实现与验证提供统一依据。其次，在工程实现层面，本人重点参与了 workflow 主链的搭建，包括任务创建与执行入口、规划结果固化、执行阶段状态推进、等待确认状态处理、人工决策生效以及总结收尾等关键流程的实现，使系统能够从自然语言任务出发，完成规划、执行、恢复和汇总的完整闭环。

在关键机制建设方面，本人重点负责候选集决策、门禁逻辑与失败恢复链路的落地。具体而言，一方面围绕 Plan、PlanPatch 和 Replan 等候选对象组织 Top-K 输出、默认建议和待确认动作，使系统在自动执行与人工审查之间具备明确切换条件；另一方面围绕运行失败、质量门禁和安全阻断等场景，将 retry、patch 和 replan 组织成分层恢复策略，并通过事件日志与任务快照保留恢复过程中的关键证据。这部分工作直接关系到系统是否具备

可控执行能力，也是本课题区别于一般工具调用型代理的重要实现内容。

此外，本人还承担了与中期报告直接相关的材料整理工作，包括验证报告、实验设计文档、项目进度梳理、图示更新以及论文参考资源归档等内容。通过将项目设计、代码实现、GitHub issue 进展和验证结果进行统一整理，可以使论文撰写不再停留于抽象描述，而是能够对应到系统当前已经具备的功能、尚待补齐的实验部分以及下一阶段的推进重点。总体而言，本人承担的模块覆盖了从设计到实现、从运行机制到论文材料的主要链路，核心目标是为课题后续的中期答辩和最终论文撰写建立一套结构清晰、证据完整的基础版本。

二 已完成的研究工作及成果

2.1 预定计划的执行情况

结合 `../thesis-project.design/plan/` 中自 2025 年 12 月底以来的阶段计划，以及当前 GitHub issue 与 GitHub Project 条目状态，可以将本课题到中期阶段为止的推进过程整理为表 2-1。从结果上看，项目的推进顺序基本遵循了“先控制面、再 workflow、再实验与论文材料”的主线：前期首先完成 FSM、HITL、PendingAction、Decision、TaskSnapshot 和 EventLog 等系统骨架，为后续真实工具接入和恢复控制打下基础；其后逐步完成 ProteinToolKG、从头设计最小闭环、Demo 可用化、CandidateSet v1、Patch/Replan 分层恢复和六阶段 workflow 覆盖；进入 3 月以后，工作重点进一步转向训练数据、增量实验、治理复核和中期论文材料固化。这种推进方式与课题本身的工程逻辑是一致的，因为如果控制面与数据契约不稳定，后续的工具接入、实验比较和论文论证都难以建立在可靠基础之上。

从表中的时间和状态可以看到，当前已经完成的部分主要集中在系统设计、控制机制、六阶段能力覆盖以及中期实验所需的数据与治理证据上，且大多数相关 issue 在 GitHub Project 中已经进入 Done。与此同时，仍有少量工作处于开放状态，主要包括横向对比实验、图表证据包定稿、多工具统一验收与阶段性总结报告等事项。这说明本课题在中期阶段的真实进度并不是“所有研究问题都已收束”，而是“系统主体和关键证据链已经完成，剩余工作主要集中在补齐比较实验、收口验收门禁和论文定稿”。

从验证证据来看，当前系统已经具备较强的中期展示支撑能力。全流程功能验证报告表明，系统已经覆盖自然语言任务解析、候选集门控、质量门控、分层 Patch/Replan 恢复、等待态回放、事件日志闭环以及总结器汇总等关键链路；可用性补充验证进一步表明，真实 LLM Provider 的 patch 与 replan 调用可返回结构化恢复结果，远端 REST 工具链可以通过本地适配器接入到执行流程中，API 端点级别的契约也已经通过测试验证。这意味着本课题在中期阶段并非停留在设计方案描述层，而是已经拥有较为明确的代码落点、测试支撑和产物沉淀。

2.2 系统总体设计

本系统的总体设计目标，是把面向蛋白质设计的复杂科研任务从“依赖人工串联多个工具与脚本”的离散流程，重构为“由显式状态机控制、由多 Agent 分工协作、由工具适

表 2-1 项目进度及毕业设计（论文）工作计划完成情况表

起始时间	完成时间	计划工作内容	备注（是否已完成）
2025.12.26	2026.01.15	完成 FSM/HITL 重构、PendingAction/Decision/TaskSnapshot/EventLog 模型落地、决策 API 与稳定版交付，对应 index(12.26-1.13).md 和 issue #9-#25 的主线任务。	已完成
2026.01.14	2026.01.20	完成 ProteinToolKG 最小可用形态、ESMFold 与 ProteinMPNN 的 KG 实例化，以及 Planner 基于 KG 的工具检索与解释能力，对应 Week 4 计划和 issue #90-#94。	已完成
2026.01.21	2026.01.25	完成从头蛋白质设计最小闭环，补齐设计任务 Plan 模板、多步执行、失败判定、总结器最小版本与 NIM 配置，对应 Week 5 计划和 issue #100-#109。	已完成
2026.02.01	2026.03.08	收敛 Demo 可用化与工具扩展，完成一键启动脚本、任务状态页、PendingAction 决策页、EventLog 时间线，以及 PLM/ProteinMPNN 远程工具接入，对应 Week 6-9 计划和 issue #120-#125、#132、#134。	已完成
2026.03.01	2026.03.15	完成 CandidateSet v1、Top-K 候选、分数/风险/成本门控、候选校验器，以及六阶段 S1/S2 覆盖，对应 Week 10 计划和 issue #138-#142、#155、#156。	已完成
2026.03.01	2026.03.15	完成 Patch/Replan 分层恢复、恢复链可观测性增强、训练数据抽取与质量门禁、SFT 数据集冻结，以及六阶段 S3/S4 覆盖，对应 Week 11 计划和 issue #143-#147、#157、#158。	已完成
2026.03.01	2026.03.15	完成 SFT/QLoRA 基线训练、双路推理与回退阈值接入，以及六阶段 S5/S6 覆盖；离线评估 issue #149 已关闭，但其 Project 条目仍显示 In Progress，说明评估基线已形成而正式发布门禁尚未完全收口。	基本完成
2026.03.14	2026.03.15	完成中期实验时间计划冻结、数据快采与版本冻结、纵向增量实验以及治理复核，对应 issue #169、#170、#171、#173，相关条目已进入 GitHub Project 的 Done。	已完成
2026.03.14	—	横向对比实验、实验章节图表证据包、多工具统一验收与三周成果报告仍在推进中，对应 issue #172、#174、#199、#200、#152；这些条目当前在 GitHub Project 中仍为 Todo。	进行中

配层统一承接执行”的连续系统。围绕这一目标，系统采用了五层组织方式：输入层负责接收自然语言目标与结构化约束；规划与控制层负责根据任务目标、工具知识图谱和治理策略生成候选计划并进行门禁判断；执行与恢复层负责实际调用工具并在失败时触发分层恢复；安全与汇总层负责风险评估和结果汇总；资源层负责沉淀工具元数据、事件日志、快照和最终产物。这样设计的核心原因在于，蛋白质设计任务本身并不是单次模型推理就能完成的线性问题，而是一个需要在规划、执行、检查、修复和人工确认之间反复切换的过程，因此系统必须拥有稳定的控制面和清晰的职责边界。

从角色划分上看，Planner 负责产生结构化候选，而不直接执行工具；Executor 负责串联计划执行、失败检测和恢复触发，但不拥有人工决策权；Safety 负责对输入、步骤和输出进行风险评估，只输出允许、告警或阻断结论；Summarizer 负责将过程性结果整理为可读报告，而不改变控制决策。这样的角色边界与当前实现是一致的：规划逻辑主要集中在 `src/agents/planner.py`，执行主链集中在 `src/agents/executor.py` 与 `src/workflow/`，等待态和决策应用则通过专门的工作流模块处理。通过这种划分，系统在工程上避免了“谁都可以决定下一步如何执行”的混乱状态，也为后续测试和审计提供了明确的责任归属。

图 2-1 采用 UML 类图形式，从课题标题所强调的“多 Agent 驱动”出发，对系统的核心协作关系、关键对象和主要依赖进行了进一步抽象。与一般仅强调工具编排的系统不同，本文系统将任务对象、 workflow 控制器、四类 Agent、工具知识图谱、适配器、待确认动作、日志快照和结果对象纳入同一张结构图中：Planner 面向“应该做什么”，Executor 面向“如何把计划执行出来”，Safety 面向“结果是否允许继续推进”，Summarizer 面向“如何把过程结果整理为对人类友好的输出”。这些 Agent 并不是并列运行的松散模块，而是通过统一的 Workflow Runtime 和状态推进机制形成有约束的协作网络。以 UML 类图表达这一关系，有助于将“对象是什么”“谁依赖谁”“哪些对象在运行期被创建和持久化”同时呈现出来，也更符合论文中对系统结构进行规范化表达的要求。

图 2-2 展示了当前系统的总体组件关系。与传统的“单个代理直接调用若干工具”不同，本文系统在接口层之下显式引入了 workflow 控制层与等待态决策层。用户或 API 提交的任务会进入统一的 Workflow Runtime，在此基础上由 Planner 结合 ToolKG、候选打分与门禁逻辑生成 Plan / PlanPatch / Replan 候选，再由 Executor 进入具体执行；一旦触发等待态，系统通过 PendingAction 和 Decision 机制暂停自动推进，并在恢复后继续运行。资源层中的 EventLog、TaskSnapshot 与报告产物则持续记录系统运行证据，使整个执行闭环既能自动完成，也能在需要时被人工回看和解释。这种设计正是本课题在中期阶段已经较为成熟的系统主线。

2.3 系统架构设计

在具体架构层面，系统目前已经形成较为清晰的分层实现。输入层主要由 FastAPI 和任务对象构成，承担用户请求接入、任务创建和基础查询能力；规划层围绕 PlannerAgent 展开，负责从任务目标中抽取意图、结合 ToolKG 选择能力路径，并生成结构化的候选计划；执行层围绕 ExecutorAgent、PlanRunner、PatchRunner 和 StepRunner 展开，承担具体步骤调度、工具调用和失败恢复；安全与汇总层通过 SafetyAgent 和 SummarizerAgent 对执

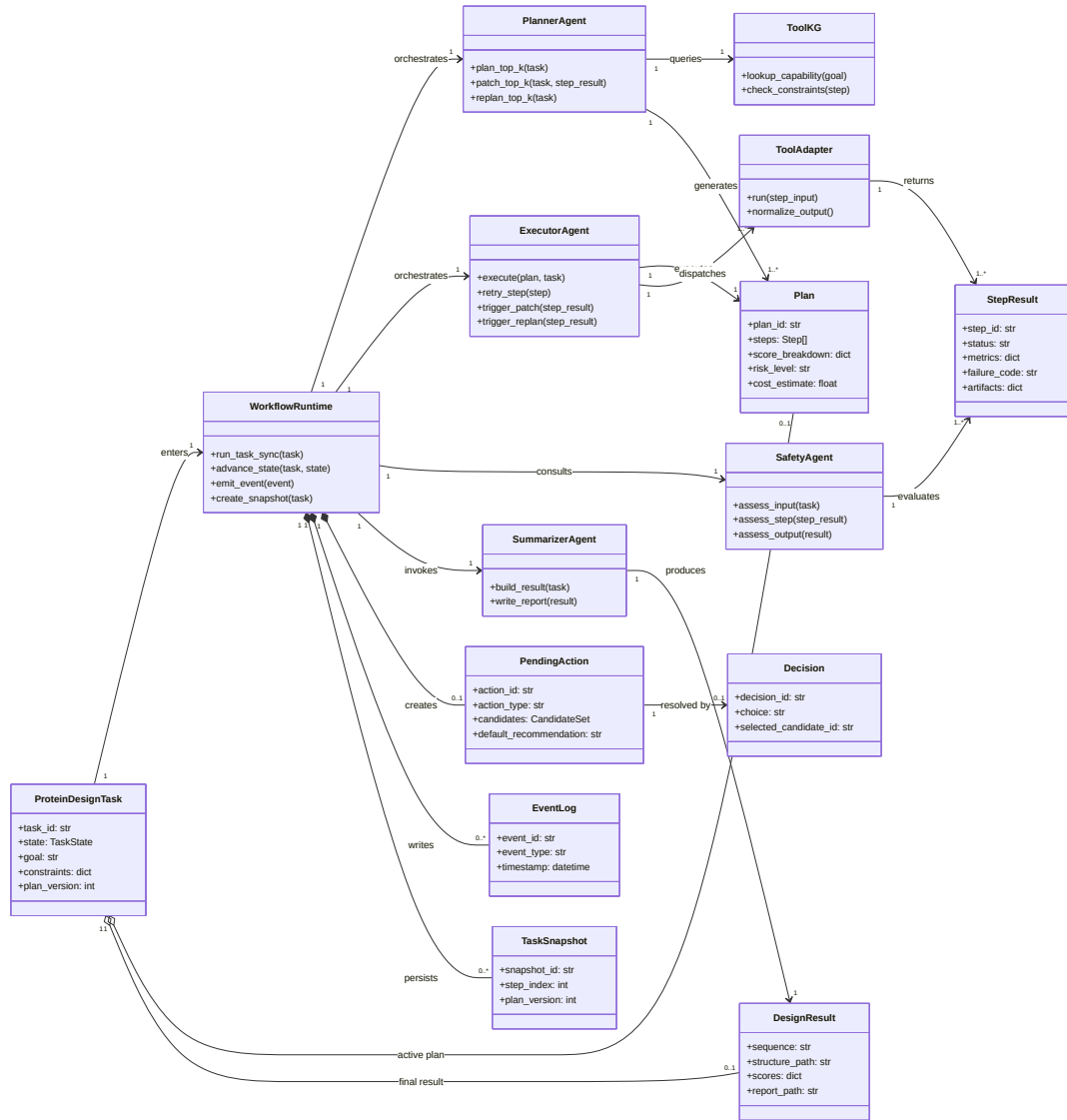


图 2-1 多 Agent 核心协作类图

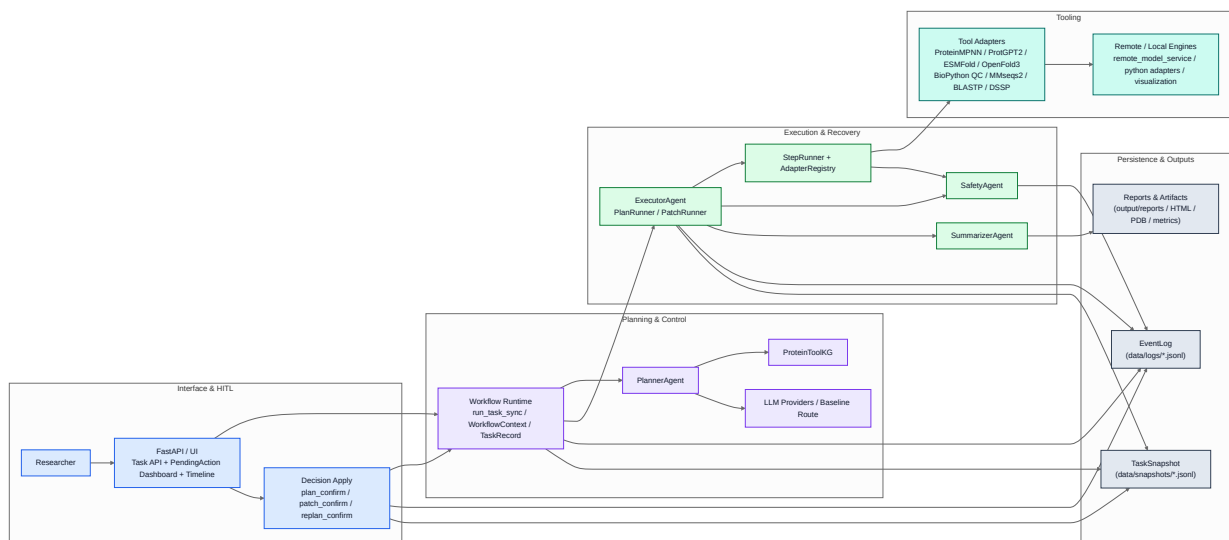


图 2-2 系统总体组件视图

行过程进行风险评估与结果整理；资源层则通过本地存储、日志、快照和输出产物目录对整个系统运行过程进行持久化。五层之间并不是简单的串行调用关系，而是通过统一数据契约、状态迁移和待确认动作保持协同。

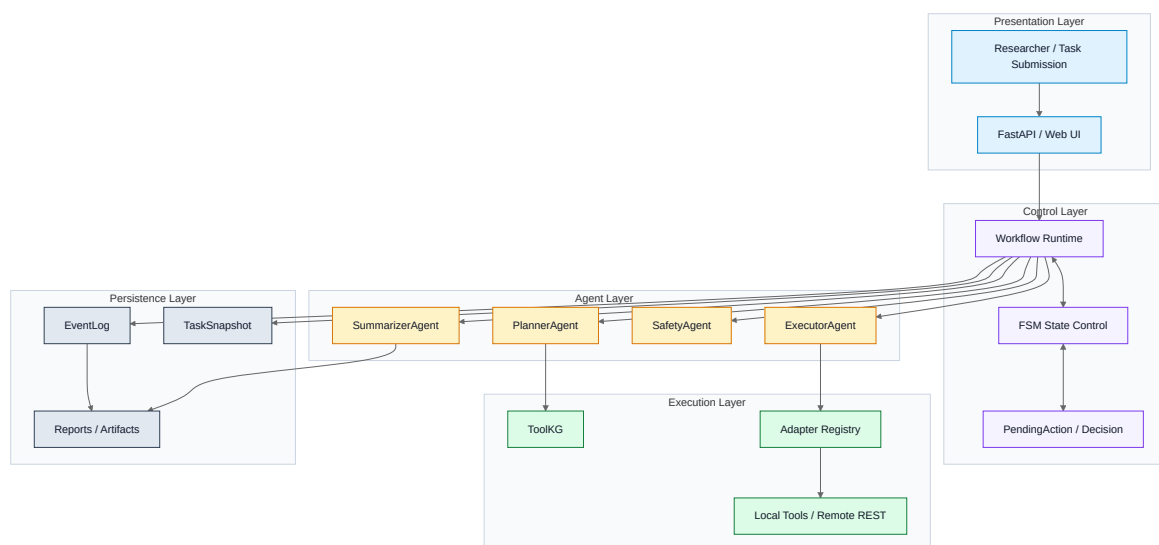


图 2-3 系统分层架构概览

图 2-3 给出了一个比详细组件图更易于快速理解的分层架构版本，适合作为论文阅读和后续答辩展示中的总览图。该图突出展示了系统从表现层、编排层、Agent 层到底层执行与持久化层的分工关系，有助于在较短时间内说明“请求如何进入系统、状态如何被控制、Agent 如何协同、工具如何被调用、证据如何被保存”这五个核心问题。相较于更细粒度的组件关系图，这种分层图更强调架构边界和责任分布，适合在论文中承担“快速建立整体认知”的功能。

系统架构中最关键的一点，是数据契约与状态语义被放在与功能实现同等重要的位置。当前系统围绕 ProteinDesignTask、Plan、StepResult、DesignResult、PendingAction

表 2-2 系统架构分层及其当前实现对应

架构层次	主要职责	当前实现对应
输入层	接收任务、组织请求、暴露查询与决策接口	src/api/main.py, 以及任务创建与 PendingAction 查询接口
规划与控制层	任务解析、ToolKG 检索、候选生成、打分与门禁	src/agents/planner.py, src/models/validation.py
执行与恢复层	按步骤执行计划、检测失败、触发 patch/replan、恢复执行	src/agents/executor.py, src/workflow/plan_runner.py, src/workflow/patch_runner.py
安全与汇总层	输入/过程/输出风险检查、结果汇总与报告生成	src/agents/safety.py, src/agents/summarizer.py
资源层	工具元数据、事件日志、快照、产物与报告持久化	src/kg/, src/storage/, output/

和 `Decision` 等对象建立了较稳定的契约，这些对象共同承担了“任务如何表示、候选如何交付、执行结果如何回写、人工决策如何生效”的问题。以 `Plan` 为例，它不仅保存步骤序列，也通过步骤输入、工具标识和元信息约束执行链如何落地；以 `StepResult` 为例，它不仅记录某一步成功或失败，还保留工具输出、度量指标、失败类型和时间戳，为后续恢复、总结和追溯提供依据；以 `DesignResult` 为例，它将最终序列、结构、得分和报告路径统一封装为面向用户和论文材料的输出对象。这些契约保证了系统运行不是依赖一系列临时变量或松散脚本，而是建立在可被校验、可被持久化的结构化对象之上。

除数据契约外，系统还将执行合法性和候选质量控制前置到了规划阶段。当前实现中，`Planner` 在生成初始 `Plan`、`Patch` 或 `Replan` 候选后，并不会简单地把结果直接交给执行器，而是先通过候选校验和打分逻辑检查工具可用性、输入输出闭包、参数合法性以及恢复容忍策略，再结合风险、成本与置信度决定是否自动放行或进入等待态。也就是说，本系统的核心算法虽然并不是本章的展开重点，但已经明确体现为“候选生成 -> 候选筛选 -> 门禁决策”的受控路径，而不是单次生成后直接执行。这样做的价值在于，一方面可以在执行前过滤掉明显不可用的候选，另一方面也使人工确认的介入具有明确触发条件，而不是依赖临时主观判断。对中期论文而言，这部分机制体现了系统架构的一个重要特点，即规划与执行不是松耦合的前后两个阶段，而是通过契约和门禁逻辑紧密衔接的整体。

2.4 模块交互设计

从任务生命周期的视角观察，当前系统的模块交互已经形成一条完整主链。用户首先通过 API 或界面提交自然语言任务，系统将其封装为统一任务对象，并进入 `CREATED` 与 `PLANNING` 状态。随后，`Planner` 根据任务目标、约束信息与 `ToolKG` 检索结果生成候选计划集合；如果门禁判断认为当前候选存在较高风险、较高成本或较低置信度，则系统会创建 `PendingAction(plan_confirm)`，进入 `WAITING_PLAN_CONFIRM`，由人工从候选集中进

行确认；如果满足自动放行条件，则系统会直接固化默认候选并推进到执行态。该过程意味着 Planner 并不直接“决定”系统下一步做什么，而是输出候选、默认建议和解释信息，真正的状态推进由 workflow 控制逻辑负责。

进入执行阶段后，Executor 会基于当前 Plan 逐步调度 StepRunner 与工具适配器完成具体步骤执行。若步骤正常完成，结果会被写入上下文并作为后续步骤输入；若步骤失败，系统首先遵循既定的重试策略，重试耗尽或检测到可修复失败类型后，再由 PatchRunner 触发局部修复候选生成。如果 Patch 候选需要人工确认，则任务进入 WAITING_PATCH_CONFIRM；若高风险或局部修复无法成立，则进一步升级到 Replan 流程，并在必要时转入再规划确认等待态。在这一过程中，Executor 负责发现问题和暂停执行，但不拥有“自行接受某个 Patch 或 Replan”的权限，这一点保证了执行控制和人工决策之间的边界清晰可见。换言之，系统在交互层面看似是一个可操作、可确认的研究辅助平台，但在控制逻辑上仍有一条清晰的亮点机制主线，即 `retry -> patch -> replan` 与 HITL 决策共同组成的分层恢复闭环。

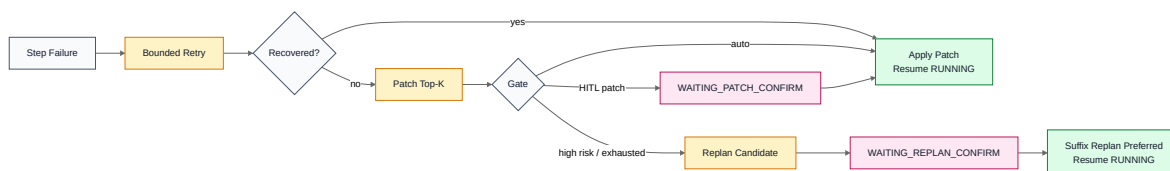


图 2-4 恢复控制与 HITL 机制概览

图 2-4 以更紧凑的方式展示了恢复控制与人在环机制之间的关系。可以看到，系统并不会在单步失败后立即进入全局重规划，而是先进行受限重试，再根据情况生成 Patch 候选并经过门禁判断；只有在 Patch 不可行、高风险或被人工拒绝时，才进一步升级到 Replan。这个过程说明，HITL 机制并不是游离在系统之外的额外确认步骤，而是和恢复控制共同构成了当前系统最重要的亮点机制之一。相较于复杂的详细时序图，这种机制图更适合在中期报告中承担“快速说明控制主线”的作用，也更方便后续直接转化为答辩展示图。

在模块交互闭环中，Safety 和 Summarizer 也承担着不可替代的作用。Safety 并不是执行结束后才进行的一次性检查，而是可以在任务输入、单步结果乃至最终输出阶段参与风险判断，输出允许、告警或阻断结论。一旦出现阻断，系统就会重新进入恢复控制链，而不是继续沿错误路径运行。Summarizer 则位于执行和展示之间，将分散在多个步骤中的结果、指标和产物路径整理为统一的 DesignResult 与报告文件，使系统输出能够同时服务于用户理解、结果归档和论文材料整理。由此可见，当前系统的模块交互设计并不是围绕“如何完成一次工具调用”展开，而是围绕“如何稳定地完成一项复杂任务”展开的。

2.5 系统的详细及实现设计

在实现层面，当前系统已经形成了较清晰的代码组织。API 入口位于 `src/api/main.py`，负责处理任务创建、任务查询、待确认动作查询、人工决策提交以及事件流展示等操作；同步运行主链通过 `run_task_sync` 串联 Planner、Executor 与 Summarizer，构成最小可用执行闭环。Planner 的实现集中在 `src/agents/planner.py`，在保留基于 ToolKG 的静态能力路径选择之外，还进一步支持 `plan_top_k`、`patch_top_k` 和 `replan_top_k` 三类候选

生成接口，并通过候选打分、默认建议与门禁判断决定系统是自动放行还是进入等待态。这里的算法亮点并未脱离系统实现单独存在，而是直接落在候选对象与评分字段中，例如 `score_breakdown`、`risk_level`、`cost_estimate` 和 `default_recommendation` 等信息都会随着候选一起进入后续控制链。Executor 的实现位于 `src/agents/executor.py`，其下通过 `PlanRunner`、`PatchRunner` 和 `StepRunner` 将执行调度、单步运行与恢复控制拆分为不同粒度的组件，以避免复杂逻辑全部堆叠在同一个模块中。

从执行控制细节来看，`PlanRunner` 负责维持计划执行的主循环，在必要时将任务从 `PLANNED` 推进到 `RUNNING`，并在执行结束后推进到 `SUMMARIZING`。`PatchRunner` 则承担失败恢复中的关键角色：它先对单步结果进行失败类型判断，再根据情况构造 `Patch` 请求、生成候选集、执行门禁判断，并决定是直接应用自动路径还是进入等待态。对于可自动执行的低风险 `Patch`，系统会经历 `WAITING_PATCH`、`PATCHING` 再回到 `RUNNING`；对于高风险或不适合局部修复的场景，则会升级为 `Replan`。与此同时，`resolve_s6_recovery_action` 等逻辑将恢复控制与六阶段 workflow 结合起来，使系统的恢复策略不只是通用异常处理，而是与蛋白质设计任务的阶段语义保持一致。换言之，核心算法中的亮点机制在代码层主要表现为三点：一是候选生成与门禁判断始终绑定出现；二是恢复控制遵循从局部到整体的递进顺序；三是再规划默认尽量保留已成功前缀，而不是无差别推翻整条链路。这样的实现方式表明，本系统并不是在发生错误后“临时加一个补丁流程”，而是把这些亮点机制直接嵌入到了正式运行主链中。

等待态与人工决策机制的实现也是当前系统的重要完成成果之一。进入等待态之前，系统会先构造 `PendingAction`，写入候选集、默认建议和解释信息，再通过事件日志与任务快照将当前上下文完整持久化，之后才允许状态切换到相应的 `WAITING_*`。这样做的直接好处是，即使系统在等待期间中断或重启，也能够基于最近一次快照恢复到一致的待确认场景。待用户提交 `Decision` 后，`decision_apply.py` 会根据动作类型将批准、重规划或取消等操作映射为后续状态推进，同时写入 `WAITING_EXIT` 与 `DECISION_APPLIED` 等结构化事件。这一实现使“人工参与”成为可被审计、可被重放的正式系统行为，而不是仅存在于界面层的外部点击动作。

在可观测与持久化方面，当前系统已经具备较为完整的证据沉淀能力。任务状态变化会同步写入事件日志；在生成新 `Plan`、应用 `Patch`、进入等待态等关键节点，系统会额外写入 `TaskSnapshot`，以保存当前计划版本、已完成步骤索引、相关产物路径以及待确认动作标识。这些数据不仅支持问题排查和过程回放，也直接服务于后续的验证报告和论文写作。结合当前已经完成的全流程功能验证、API 端点验证、真实 `Provider` 可用性验证以及远端 `REST` 工具链接入验证，可以认为系统的详细实现已经超出了“功能演示原型”的范围，开始具备面向实际科研工作流继续演进的基础条件。

最后，从当前代码与验证材料的对应关系来看，本课题已经形成了较完整的“设计文档、代码实现、测试验证、报告沉淀”四位一体结构。设计文档给出了状态机、角色边界、候选契约和恢复顺序等系统约束；代码实现将这些约束落实到 `API`、`workflow`、`适配器` 和 `存储模块` 中；测试与验证报告则进一步证明这些机制并非纸面设计，而是已经能够在实际运行中形成闭环。对于中期论文而言，这种一致性具有重要意义，因为它说明当前章节中的介绍并不是抽象设想，而是与现有项目仓库中的真实实现保持一致的阶段成果。

三 后期拟完成的研究工作及进度安排

3.1 后期拟完成的工作

本节建议聚焦尚未完成但已经明确规划的内容，例如：

- 横向对比实验 E0 / E1 / E2 的运行与整理；
- 多工具统一验收与自动化门禁完善；
- 论文图表、实验结果和正式定稿；
- 对离线门禁阻断项进行补样与复核。

3.2 后期工作的进度安排

本节建议按时间轴列出后续工作计划，推荐至少包括：

- 实验补齐阶段；
- 论文写作与图表整理阶段；
- 答辩准备与模板定稿阶段。

四 存在的问题与困难

本节建议如实陈述当前阶段的限制与风险，不宜回避未完成项。可以结合项目现状重点说明：

- 横向对比实验尚未完成；
- 离线门禁仍存在未达标指标；
- 部分真实远端工具链受外部平台条件限制；
- 主实验中的 WAITING 决策样本仍需补充。

五 论文按时完成的可能性

本节建议从以下方面给出按时完成论文的判断依据：

- 系统主体架构与实现已经完成；
- 中期模板、章节骨架和参考资料已经整理；
- 关键验证报告与展示材料已经具备；
- 剩余工作主要集中在实验补齐与写作收尾，边界清晰。

建议在结尾明确说明：虽然仍有横向实验和门禁收尾工作，但整体项目已经具备继续推进到论文定稿阶段的基础。